Windows API for Red Team #101

Author

Joas Antonio dos Santos

https://www.linkedin.com/in/joas-antonio-dos-santos/

TABELA DE CONTEÚDO

01 O QUE É WINDOWS API

07 O QUE É TABELA IAT

14 TABELA IAT E ORDINAIS

ESCRITO POR JOAS A SANTOS

O QUE É WINDOWS API

INTRODUÇÃO

API do Windows, conhecida também como Windows API ou WinAPI, é um conjunto de interfaces de programação de aplicações (APIs) disponibilizadas pela Microsoft para permitir a interação entre programas e o sistema operacional Windows. Essas APIs fornecem funções que permitem aos desenvolvedores manipular componentes do sistema operacional, como janelas, arquivos, processos, gráficos, redes, entre outros.

A WinAPI é dividida em várias seções, cada uma lidando com diferentes aspectos do sistema operacional:

1. **User Interface**: APIs para manipulação de janelas, menus, caixas de diálogo e outros elementos de interface com o usuário.
2. **Graphics Device Interface (GDI)**: APIs que permitem a manipulação de gráficos, como desenho de formas, textos e imagens.
3. **System Services**: Funções que fornecem acesso a recursos mais profundos do sistema operacional, como gerenciamento de arquivos, eventos e processos.
4. **Multimedia**: APIs dedicadas ao manuseio de áudio e vídeo.
5. **Networking**: Funções para gerenciar comunicações de rede.

Essas APIs são essenciais para o desenvolvimento de aplicações nativas para Windows e são usadas em uma vasta gama de software, desde aplicativos simples até sistemas operacionais completos que rodam sobre a plataforma Windows. Elas estão disponíveis para várias linguagens de programação, como C, C++, e também podem ser acessadas através de outras linguagens via "wrappers" ou bibliotecas específicas.

Funcionalidades

As APIs do Windows envolvem uma série de funcionalidades que permitem aos desenvolvedores criar aplicações que interagem diretamente com o sistema operacional Windows. Essas APIs cobrem uma ampla gama de funcionalidades,

desde a criação e gestão de elementos gráficos da interface do usuário até o acesso a recursos do sistema, como arquivos, redes e dispositivos. Aqui estão alguns dos principais componentes e funcionalidades que as APIs do Windows envolvem:

1. **Gerenciamento de Interface do Usuário**:
    - Criação e manipulação de janelas e diálogos.
    - Processamento de mensagens e eventos.
    - Controles de interface como botões, caixas de texto e listas.
2. **Graphics Device Interface (GDI)**:
    - Desenho de gráficos na tela.
    - Manipulação de fontes e textos.
    - Renderização de imagens e formas.
3. **DirectX**:
    - APIs para desenvolvimento de jogos e aplicativos que requerem gráficos de alta performance.
    - Suporte para áudio, vídeo e processamento de eventos em tempo real.
4. **System Services**:
    - Acesso a arquivos e diretórios.
    - Gerenciamento de memória e processos.
    - Serviços de segurança e controle de acesso.
5. **Comunicação e Networking**:
    - APIs para protocolos de rede, como TCP/IP.
    - Funções para criação e gerenciamento de conexões de rede.
    - Serviços de comunicação entre processos.
6. **Multimedia**:
    - Manipulação de arquivos de áudio e vídeo.
    - Captura e reprodução de mídia.
7. **Windows Registry**:

- Acesso e manipulação do registro do Windows, que armazena configurações do sistema e aplicativos.

8. **COM e DCOM**:

    - Component Object Model (COM) permite a interação entre componentes de software que podem estar em processos ou mesmo computadores diferentes.

    - Distributed COM (DCOM) estende essas capacidades para redes.

9. **Windows Runtime (WinRT)**:

    - Uma API mais recente que permite desenvolvimento de aplicações para a plataforma Windows com suporte a múltiplos dispositivos, incluindo computadores, tablets e telefones.

As APIs do Windows são projetadas para oferecer uma plataforma robusta para desenvolvimento de software, permitindo que os aplicativos aproveitem os recursos e capacidades do hardware e do sistema operacional de maneira eficiente e segura.

Como trabalhar com API do Windows?

Para acessar as APIs do Windows, você pode seguir alguns passos básicos que envolvem a escolha de uma linguagem de programação adequada, a configuração do ambiente de desenvolvimento, e a utilização de bibliotecas específicas que permitem a interação com o sistema operacional. Aqui estão as etapas detalhadas:

1. **Escolha uma Linguagem de Programação**:

    - As APIs do Windows são tradicionalmente acessadas usando C ou C++, mas também podem ser utilizadas através de outras linguagens como C#, Visual Basic, e Python, usando bindings ou wrappers apropriados.

2. **Configure seu Ambiente de Desenvolvimento**:

    - **Para C/C++**: Instale um ambiente de desenvolvimento integrado (IDE) como Microsoft Visual Studio, que oferece suporte completo para desenvolvimento Windows com C/C++. Visual Studio já inclui os cabeçalhos e bibliotecas necessários para acessar as Windows APIs.

    - **Para C# ou Visual Basic**: O Visual Studio também é recomendado, pois oferece acesso fácil ao .NET Framework e ao Windows Runtime, que são interfaces mais modernas para as APIs do Windows.

    - **Para Python**: Instale uma biblioteca como **pywin32**, que oferece acesso a muitas das APIs do Windows.

3. **Aprenda a API Específica**:

    - Consulte a documentação oficial da Microsoft para entender as funções específicas da API que você deseja usar. A documentação da Microsoft é bastante abrangente e inclui exemplos de código.

    - A Microsoft Docs (https://docs.microsoft.com/) é o recurso recomendado para encontrar informações detalhadas sobre cada API.

4. **Pratique com Exemplos**:

    - Comece com exemplos simples, como criar uma janela ou manipular arquivos, e gradualmente avance para tarefas mais complexas.

    - A comunidade de desenvolvedores e fóruns online, como Stack Overflow, podem ser recursos úteis para aprender e resolver problemas específicos.

5. **Use Bibliotecas e Frameworks**:

    - Para tarefas comuns, existem muitas bibliotecas e frameworks que simplificam o uso das APIs do Windows. Por exemplo, o .NET Framework para C# e VB ou o Qt para C++ oferecem abstrações de alto nível que facilitam o desenvolvimento de aplicações robustas.

6. **Compilação e Linkagem**:

    - Certifique-se de que seu projeto está corretamente configurado para compilar e linkar as bibliotecas necessárias para acessar as APIs do Windows. No Visual Studio, isso geralmente é gerenciado automaticamente, mas em outros ambientes, pode ser necessário configurar manualmente.

Requisitos mínimos de Hardware:

- Windows 10 latest version (here)

- 8 GB Memory Ram and 120 GB Disk Minimum

- Visual Studio 2022 Community Edtion (here)

Começar com um bom ambiente de desenvolvimento e acessar recursos educativos adequados são fundamentais para aproveitar efetivamente as APIs do Windows em seus projetos de software.

API do Windows no contexto de ataque

As APIs do Windows são fundamentais no contexto de segurança cibernética, particularmente em atividades de Red Team e simulação de adversários. A capacidade dessas APIs de interagir profundamente com o sistema operacional Windows permite que os testadores de penetração e simulações de adversários executem uma variedade de táticas avançadas, que podem incluir leitura da memória de outros processos, execução de código com privilégios elevados, enumeração detalhada de sistemas, e neutralização de soluções antivírus (AV). Abaixo, detalho por que essas capacidades são cruciais para a segurança cibernética e a importância de se preocupar com elas:

1. **Execução de Código com Privilégios Elevados**:

    - O controle de privilégios é uma parte crítica da segurança do sistema. As APIs do Windows permitem que programas executem código com privilégios elevados, o que pode ser explorado para obter controle total sobre o sistema. Este é um vetor de ataque essencial que os Red Teams testam para garantir que os sistemas estão adequadamente protegidos contra escalonamentos de privilégio.

2. **Leitura de Memória de Outros Processos**:

    - A capacidade de ler a memória de outros processos é uma ferramenta poderosa para adversários, pois pode revelar informações sensíveis, incluindo senhas e chaves de criptografia. Red Teams usam essa técnica para simular ataques que tentam extrair dados sensíveis que podem ser mal utilizados.

3. **Enumeração de Sistema**:

    - As APIs do Windows podem ser usadas para enumerar recursos e configurações do sistema, permitindo que os adversários mapeiem o ambiente de destino. Isso é fundamental para o planejamento de ataques mais direcionados e eficazes, ajudando Red Teams a identificar potenciais vulnerabilidades e configurações incorretas.

4. **Neutralização de Antivírus**:

    - Eliminar ou desativar soluções de AV é uma técnica comum entre os adversários para evitar detecção e análise. Usando APIs do Windows, Red Teams podem testar a resiliência das soluções de segurança implementadas contra táticas que tentam desabilitar ou contornar essas proteções.

5. **Personalização e Discrição**:

    - Ao usar código personalizado que chama diretamente a API do Windows, os Red Teams podem criar ferramentas que são altamente adaptadas ao ambiente de destino e menos propensas a serem detectadas por soluções de segurança convencionais. Isso simula adversários avançados que usam código personalizado e técnicas de evasão sofisticadas.

O estudo e a compreensão das APIs do Windows são essenciais para as equipes de segurança, pois oferecem a habilidade de não apenas proteger contra, mas também simular, táticas avançadas usadas em ciberataques reais.

O QUE É TABELA IAT?

INTRODUÇÃO

A tabela de Import Address Table (IAT) é uma componente crucial em arquivos executáveis no formato Portable Executable (PE), que é usado em sistemas operacionais Windows. A IAT é utilizada para gerenciar chamadas de funções que são importadas de outros módulos ou bibliotecas dinâmicas (DLLs). Quando um executável ou DLL precisa de uma função que está em outra DLL, essa função é chamada através de uma entrada na IAT, que contém os endereços de todas as funções importadas que o executável ou DLL precisa.

Qual a sua Importância?

A tabela IAT é essencial por várias razões no desenvolvimento e na segurança de software:

1. **Resolução Dinâmica de Endereços**: A IAT permite que os endereços das funções importadas sejam resolvidos em tempo de execução pelo loader do Windows. Isso significa que o código executável pode usar várias versões de uma DLL sem precisar ser recompilado, desde que as interfaces das funções permaneçam consistentes.

2. **Otimização e Manutenção**: Manter as chamadas de função através da IAT facilita a manutenção e atualização de aplicações, permitindo que novas versões de DLLs sejam simplesmente substituídas sem alterar o código base principal.

3. **Segurança**: A IAT é frequentemente um alvo em técnicas de exploração de software e malware. Modificar a IAT pode permitir que um atacante redirecione chamadas de função para código malicioso. Por isso, entender e proteger a IAT é fundamental para a segurança do software.

Usando PEViewer para Visualizar os Imports

PEView é uma ferramenta que permite aos usuários visualizar e analisar os componentes internos de arquivos executáveis no formato PE, incluindo a Import Address Table. Aqui está como usar PEView para examinar a IAT de um executável:

1. **Baixar e Abrir PEView**: Primeiro, é necessário baixar e abrir o PEView. Carregue o arquivo PE (executável ou DLL) que você deseja analisar.

2. **Navegar até a Seção de Imports**: Dentro do PEView, navegue até a seção que lista os imports. Esta seção mostrará todas as DLLs das quais o arquivo depende, bem como as funções específicas que são importadas de cada uma.

3. **Examinar a IAT**: Ao selecionar uma DLL específica, você pode ver a lista de funções importadas que estão associadas a essa DLL. Estas entradas correspondem aos endereços na IAT, onde o loader do sistema resolverá os endereços reais das funções em tempo de execução.

4. **Análise de Segurança**: Utilize esta visualização para entender quais funções seu aplicativo está importando e de quais bibliotecas. Isso pode ser crucial para identificar possíveis vulnerabilidades, como o uso de funções desatualizadas ou inseguras.

O uso do PEView é uma prática comum entre desenvolvedores e analistas de segurança para entender melhor as dependências de aplicativos Windows e para verificar a integridade dos imports em software, sendo uma ferramenta valiosa tanto para desenvolvimento quanto para auditoria de segurança.

Passo a passo:

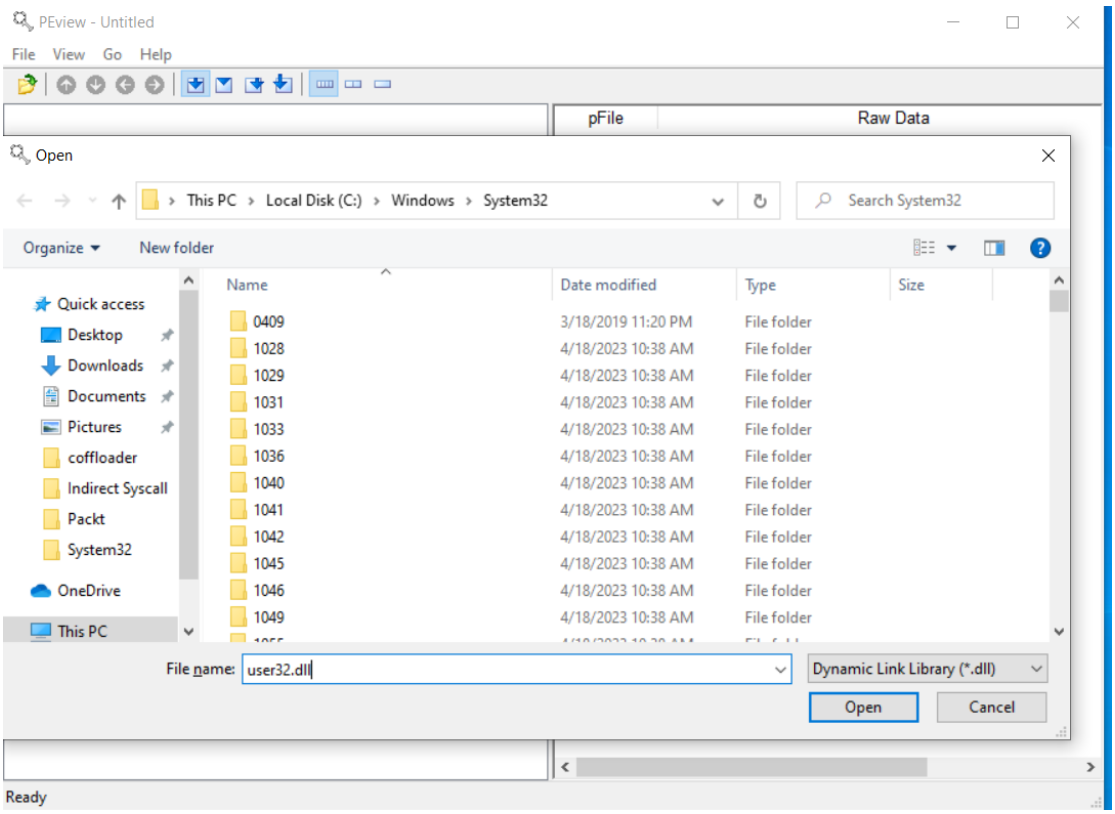

Figura 1 – Selecionando a DLL para analisar os imports

Após abrir o PEView, selecione uma DLL, nesse caso vou importar o user32.dll que contém inúmeras chamadas de API.

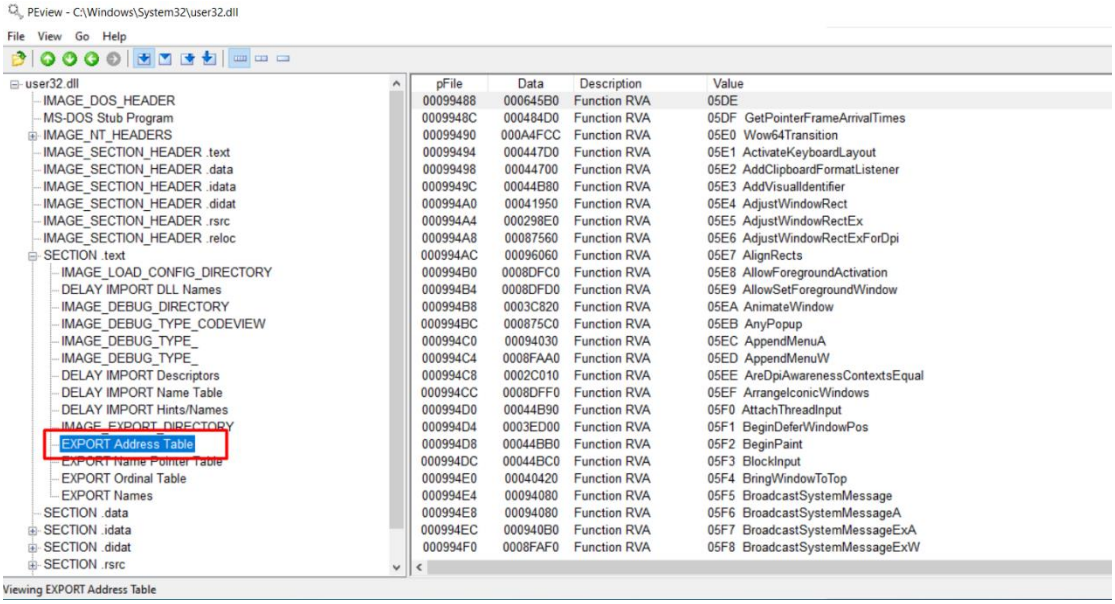

Figura 2 – Analisando as tabelas de endereços

Ao acessarmos as tabelas de endereços, vamos encontrar inúmeras funções de API que a dll possui em sua tabela, assim conseguimos obter informações cruciais como endereços de memória dessas funções.

Podemos criar um script em python que faça a extração desses endereços de um arquivo PE.

Para extrair a tabela Import Address Table (IAT) de arquivos PE (Portable Executable) em Python, você pode utilizar a biblioteca **pefile**. Esta biblioteca é amplamente usada para análise e manipulação de arquivos PE em ambientes de segurança e engenharia reversa.

Aqui está um script básico que demonstra como extrair e imprimir a tabela IAT de um arquivo PE usando **pefile**:

```
import pefile

import sys


def extract_iat(pe_file):

    try:

        # Load the PE file

        pe = pefile.PE(pe_file)


        # Check if the file has an imports table

        if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):

            print(f"IAT for {pe_file}:")


            # Iterate over each entry in the imports table

            for entry in pe.DIRECTORY_ENTRY_IMPORT:

                print(f"Imports from {entry.dll.decode()}:")

                for imp in entry.imports:

                    address = hex(imp.address)

                    name = imp.name.decode() if imp.name else 'Ordinal Import'

                    print(f"    {address} {name}")
```

```
        else:
            print("No imports found.")


    except Exception as e:
        print(f"Error processing the file: {e}")


if __name__ == '__main__':
    if len(sys.argv) > 1:
        extract_iat(sys.argv[1])
    else:
        print("Please provide the path to the PE file as an argument.")
```

Instalar a biblioteca pefile: Antes de rodar o script, você precisa instalar a biblioteca **pefile**. Você pode fazer isso usando pip:

```
pip install pefile

python extract_iat.py caminho_para_o_arquivo.exe
```

Funcionamento do Script

- O script carrega o arquivo PE usando **pefile.PE()**.
- Verifica se o arquivo possui uma tabela de importações (**DIRECTORY_ENTRY_IMPORT**).
- Para cada DLL na tabela de importações, ele lista as funções que estão sendo importadas, incluindo seus endereços.

Este script é uma introdução básica e pode ser expandido ou modificado para incluir mais funcionalidades, como filtrar por tipos específicos de importações ou manipular os dados de outras maneiras conforme necessário.

```
C:\Users\Operator\Desktop\Scripts\WIndows API 101>python iatextract.py C:\Windows\System32\user32.dll
IAT for C:\Windows\System32\user32.dll:
Imports from win32u.dll:
    0x1800889a8 NtUserGetClipboardFormatName
    0x1800889b0 NtUserRegisterWindowMessage
    0x1800889b8 NtUserGetKeyNameText
    0x1800889c0 NtUserMapVirtualKeyEx
    0x1800889c8 NtUserEnumDisplayDevices
    0x1800889d0 NtUserGetClassInfoEx
    0x1800889d8 NtUserChangeDisplaySettings
    0x1800889e0 NtUserRemoveProp
    0x1800889e8 NtUserUnregisterClass
    0x1800889f0 NtUserEnumDisplaySettings
    0x1800889f8 NtUserGetAltTabInfo
    0x180088a00 NtUserSetClassLong
    0x180088a08 NtUserGetMessage
    0x180088a10 NtUserGetKeyboardLayoutName
    0x180088a18 NtUserDrawCaptionTemp
    0x180088a20 NtUserSetProp
    0x180088a28 NtUserVkKeyScanEx
    0x180088a30 NtUserCallMsgFilter
    0x180088a38 NtUserCallHwndLockSafe
    0x180088a40 NtUserSetImeOwnerWindow
    0x180088a48 NtUserNotifyIMEStatus
    0x180088a50 NtUserUpdateInputContext
    0x180088a58 NtUserCountClipboardFormats
    0x180088a60 NtUserGetPriorityClipboardFormat
    0x180088a68 NtUserGetClipboardOwner
    0x180088a70 NtUserGetClipboardSequenceNumber
```

Figura 3 – Execução do script de extração da tabela IAT

Selecione o PE que deseja extrair os endereços de IAT, mas certifique-se de que seja um arquivo válido, antes tente abrir em um visualizador de PE para obter informações mais detalhadas, depois disso você pode utilizar esses scripts caso queira trabalhar com técnicas mais avançadas para trabalhar com uma função específica.

Download do PEView: http://wjradburn.com/software/

Download extract_iat: https://github.com/CyberSecurityUP/Windows-API-for-Red-Team/blob/main/extract_iat.py

Tabela IAT no contexto de ataque

As técnicas que envolvem a tabela Import Address Table (IAT) na evasão de detecção e no desenvolvimento de malware são sofisticadas e abrangem vários métodos para manipular como os programas acessam e executam código externo. Essas técnicas são essenciais para entender tanto para desenvolvedores de segurança quanto para profissionais de TI focados em defesa cibernética. Aqui estão algumas das técnicas mais relevantes:

1. Uso de Ordinals

No contexto das DLLs, além de importar funções pelo nome, é possível importar funções pelo ordinal, que é basicamente um índice associado a cada função em uma DLL. O uso de ordinais em vez de nomes de funções pode complicar a análise e detecção de malware, pois observadores externos precisam saber quais funções correspondem a quais ordinais em cada DLL, o que pode variar entre diferentes versões da mesma DLL.

2. Syscalls

Syscalls (chamadas de sistema) são outro método de baixo nível utilizado por malwares para interagir diretamente com o sistema operacional, contornando as APIs padrão do Windows e, consequentemente, a IAT. Ao fazer syscalls diretamente, o malware pode evitar detecções baseadas no monitoramento de chamadas de API comuns, já que não depende das importações listadas na IAT.

3. Hooks

O "hooking" é uma técnica que envolve a interceptação de chamadas de função ou mensagens/eventos do sistema. Um malware pode usar hooking na IAT para redirecionar chamadas de funções legítimas para funções maliciosas sem alterar o código do programa host. Isso pode ser usado para capturar dados, modificar comportamentos de programas ou desabilitar funcionalidades de segurança.

4. Unhooks

Para evadir a detecção por soluções de segurança que utilizam hooking para monitorar o comportamento de programas, alguns malwares podem executar "unhooks". Essa técnica envolve a restauração das entradas originais da IAT para seus estados pré-hook, temporariamente durante a execução de atividades maliciosas, para parecer menos suspeito para ferramentas de monitoramento.

5. Patching da IAT

Malwares podem modificar a IAT de um processo em execução para apontar para suas próprias funções maliciosas em vez das bibliotecas originais. Isso permite que o malware intercepte e manipule dados ou comportamentos do programa infectado.

Proteção e Detecção

Para se proteger contra essas técnicas, os sistemas de defesa devem implementar monitoramento e análise de comportamento em tempo de execução, verificação de integridade da IAT, e uso de tecnologias anti-hooking. Além disso, a análise forense e a engenharia reversa continuam sendo ferramentas valiosas para entender como os malwares utilizam essas técnicas e desenvolver medidas de proteção eficazes.

TABELA IAT E ORDINAIS

INTRODUÇÃO

Os ordinais são uma forma alternativa de referenciar funções ou variáveis exportadas por uma biblioteca dinâmica de vínculo (DLL) no Windows. Em vez de usar os nomes das funções, que é o método mais comum e legível, os ordinais utilizam números inteiros sequenciais como identificadores para as funções exportadas.

Como Funcionam os Ordinais

Quando uma DLL é criada, o compilador ou o linker pode atribuir a cada função exportada um número ordinal único dentro daquela DLL. Este número é um índice baseado em zero ou um que não necessariamente segue a ordem alfabética dos nomes das funções. O desenvolvedor também pode especificar manualmente esses números se desejar controlar o esquema de numeração dos ordinais.

As funções exportadas podem então ser importadas por outros módulos (executáveis ou outras DLLs) usando esses ordinais. Isso elimina a necessidade de usar strings de nomes, o que pode resultar em um pequeno ganho de performance no processo de carregamento da aplicação, já que a correspondência de strings é mais custosa do que comparar números inteiros.

Vantagens do Uso de Ordinais

1. **Eficiência**: A busca por um índice numérico é geralmente mais rápida do que por uma string de nome de função. Isso pode melhorar o tempo de carregamento do software em situações onde o desempenho de carregamento é crítico.

2. **Obfuscação**: O uso de ordinais pode servir como uma forma de obfuscação, tornando mais difícil para os analistas entenderem o

propósito de uma função sem uma análise mais profunda, já que o nome descritivo da função não está disponível.

Desvantagens e Riscos

1. **Manutenibilidade**: Usar ordinais pode dificultar a manutenção do código, já que revisar ou atualizar o código para usar ou não certas DLLs se torna mais complexo sem os nomes das funções para referência.

2. **Compatibilidade**: Se uma DLL é atualizada e os ordinais são alterados (por exemplo, funções são adicionadas ou removidas), isso pode quebrar a compatibilidade com aplicativos existentes que esperam uma função específica em um ordinal específico.

Ordinais na Segurança e Malware

Em segurança cibernética, os ordinais são frequentemente usados em técnicas de malware para dificultar a análise e detecção. Por exemplo, um malware pode importar funções críticas usando ordinais para esconder suas intenções verdadeiras e evitar detecções baseadas em análises de strings de importação conhecidas por serem maliciosas.

Exemplo prático:

Vamos ver um simples exemplo de como podemos trabalhar com ordinals, vou utilizar uma API do Windows chamada MessageBoxA

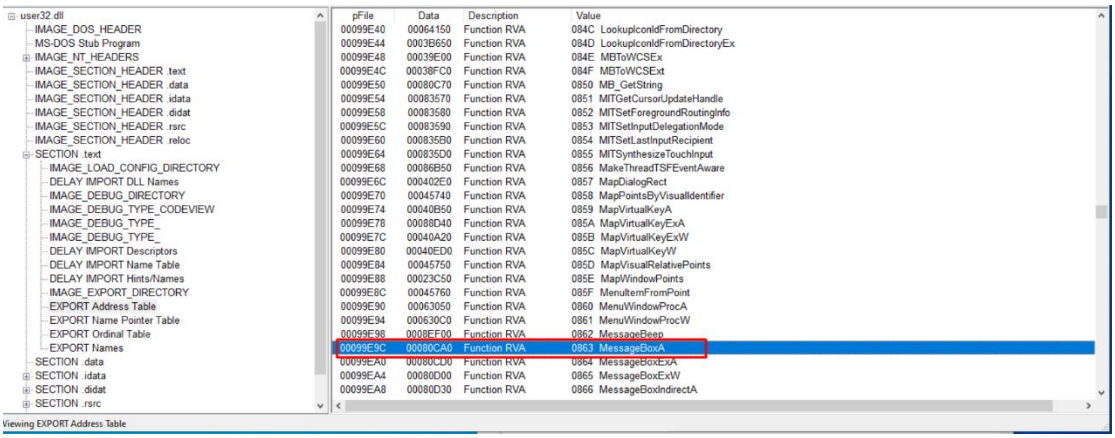

Figura 4 – Com a tabela IAT, selecione a função desejada

Nesse caso eu escolhi o MessageBoxA. Pois cada função exportada por uma DLL pode ser identificada tanto por um ordinal numérico quanto por um nome. As funções também podem ser importadas de uma DLL usando esses ordinais ou nomes. O ordinal indica a posição do ponteiro da função na

tabela de endereços de exportação da DLL. É comum que funções internas sejam exportadas somente por ordinais.

Após selecionarmos a função, vamos usar um script para extrair o ordinal de uma função.

Download:

https://github.com/CyberSecurityUP/Windows-API-for-Red-Team/blob/main/ordinal_pe.py

```
C:\Users\Operator\Desktop\Scripts\WIndows API 101>python ordinalspe.py C:\Windows\System32\user32.dll MessageBoxA
Function 'MessageBoxA' has ordinal: 2150 (Decimal)

C:\Users\Operator\Desktop\Scripts\WIndows API 101>
```

Figura 5 – Resultado do Ordinal da função MessageBoxA

Um detalhe interessante é que apesar de o decimal ter dado 2150, pode ocorrer de não ser o ordinal exato, assim sempre pule 2 casas para mais ou para menos, por exemplo: Resultado foi 2150, teste 2148 ou 2152.

Agora vamos escrever nosso código:

Vou utilizar a linguagem C++, contudo gosto sempre de adaptar o mesmo códigos em outras linguagens como Rust, C# (PInvoke and DInvoke), Python e Golang.

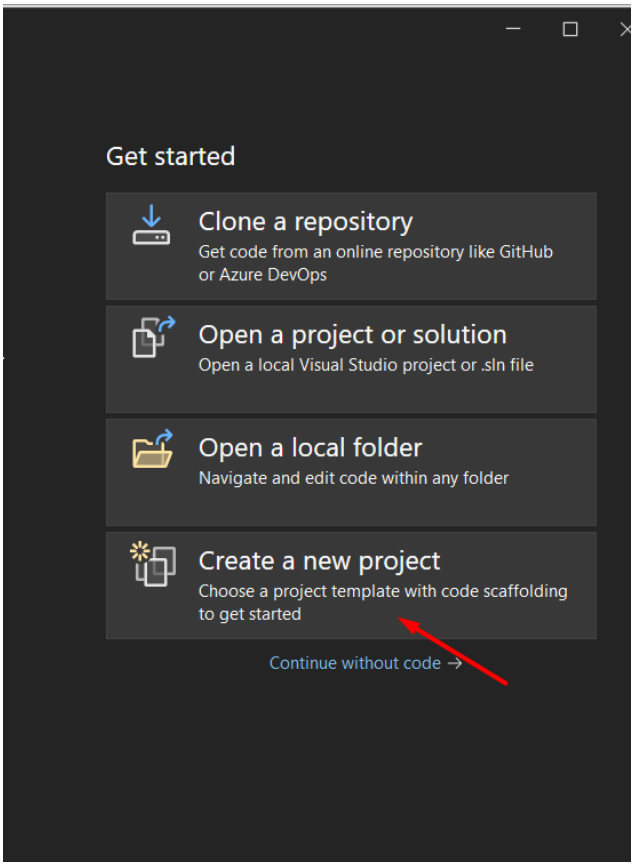

Figura 6 – Crie um projeto

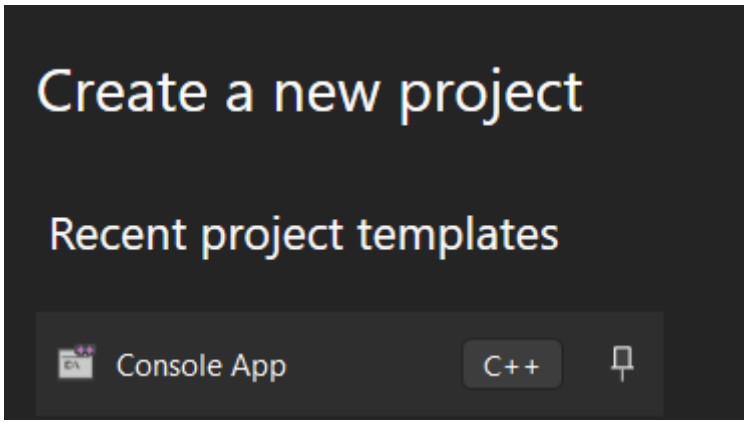

Figura 7 – Selecione um template chamado Console APP C++

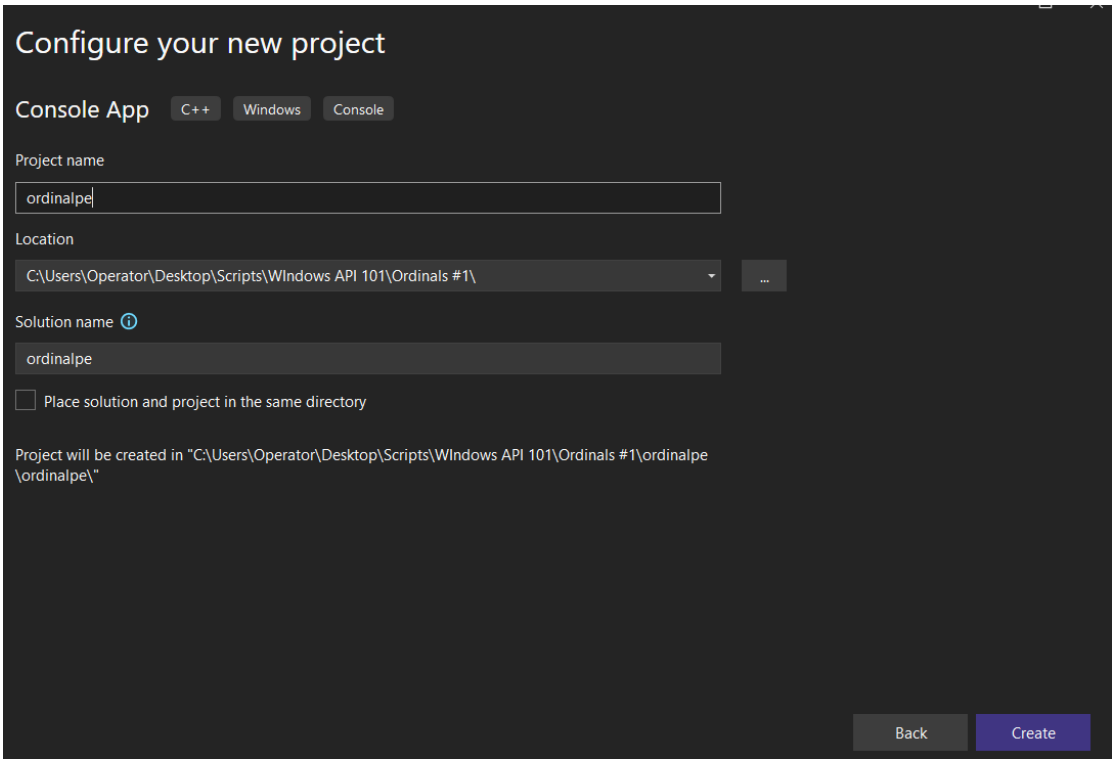

Figura 8 – Defina um nome para o seu projeto

Após criar o projeto, você pode trabalhar com o exemplo abaixo conforme o ordinal que você obteve no seu MessageBoxA

Escrevendo o nosso código:

```
#include <windows.h>
#include <iostream>
#include <stdio.h>


int main() {

    HMODULE hModule = LoadLibrary(L"user32.dll");
    if (!hModule) {
        std::cerr << "Failed to load user32.dll!" << std::endl;
        return 1;
    }
```

- **windows.h**: Inclui a API do Windows, necessária para funções como **LoadLibrary** e **GetProcAddress**.
- **iostream**: Permite a utilização de entradas e saídas de dados em C++, neste caso usado para imprimir mensagens de erro.
- **stdio.h**: Normalmente usado para entrada e saída padrão. No código fornecido, não é explicitamente utilizado e poderia ser removido.
- **Int main:** Este é o ponto de entrada do programa, onde a execução começa.

- **LoadLibrary:** Carrega a DLL especificada (user32.dll neste caso), que contém muitas das funções básicas da interface do usuário do Windows, incluindo MessageBoxA. *Se LoadLibrary falhar em carregar a DLL, uma mensagem de erro é impressa e o programa termina com o código de erro 1.*

- **HMODULE:** Tipo de handle usado para carregar módulos; hModule é um handle para o módulo carregado.

Obter o endereço da função pelo Ordinal

```
    typedef int (WINAPI* MsgBoxFunc)(HWND, LPCSTR, LPCSTR, UINT);
    MsgBoxFunc OrdinalBoxA = (MsgBoxFunc)GetProcAddress(hModule,
(LPCSTR)2150);

    if (!OrdinalBoxA) {
        std::cerr << "Failed to locate the function!" << std::endl;
        FreeLibrary(hModule);
        return 1;
    }

    OrdinalBoxA(NULL, "Hello, World!", "Test MessageBoxA", MB_OK |
MB_ICONINFORMATION);

    FreeLibrary(hModule);
    return 0;

}
```

- **GetProcAddress**: Obtém o endereço da função na DLL carregada, especificada pelo ordinal **2150**. Este número é o identificador numérico da função **MessageBoxA**.

- **typedef**: Define **MsgBoxFunc** como um ponteiro para função do tipo **MessageBoxA**.

- Se a função não for encontrada (ou seja, **OrdinalBoxA** é **nullptr**), imprime uma mensagem de erro, libera a DLL carregada e termina o programa com erro.

- **OrdinalBoxA:** Uma vez que o endereço foi recuperado com sucesso, a função MessageBoxA é chamada usando o ponteiro para a função.

- Os parâmetros passados são NULL para o handle da janela (janela pai), a mensagem "Hello, World!", o título "Test MessageBoxA" e os flags MB_OK | MB_ICONINFORMATION para o tipo de botões e ícone a serem exibidos na caixa de mensagem.

- **FreeLibrary:** Libera o módulo carregado, neste caso, user32.dll.

- **return 0;:** Termina a execução do programa normalmente, indicando que não houve erros.

- Eu poderia ocultar também as funções MessageBoxA e user32.dll para que não sejam listadas na tabela IAT também

Agora vamos ver qual oo resultado disso tudo no final

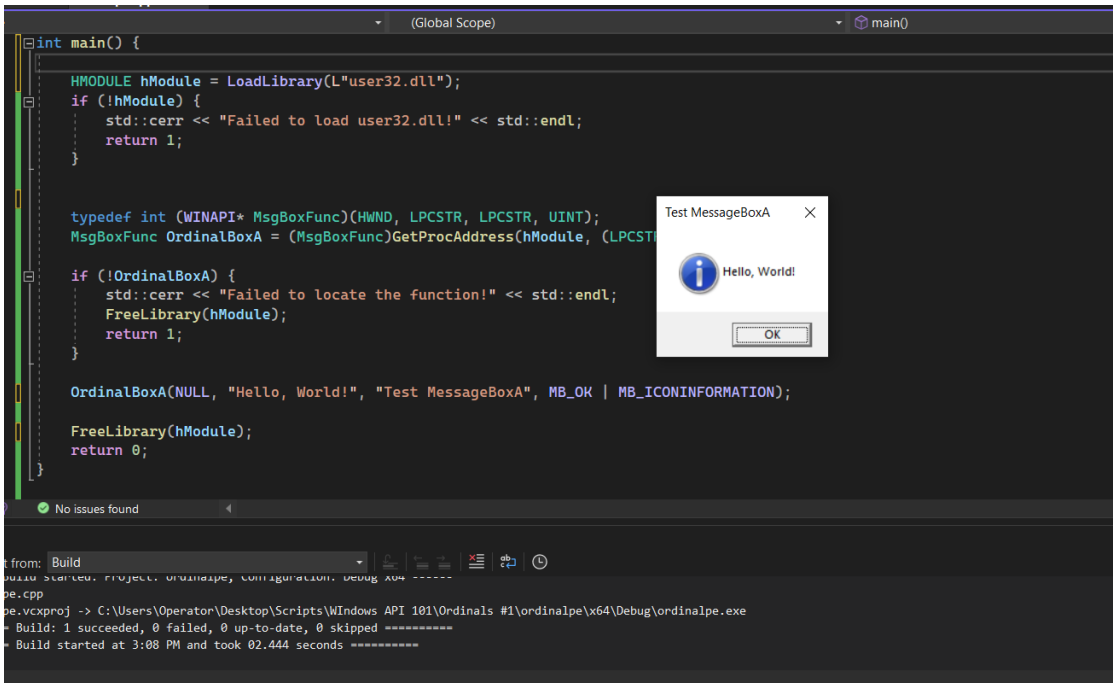

Figura 9 – Resultado da execução do código

Temos o nosso Hello World usando um ordinal, em vez de utilizarmos a biblioteca MessageBoxA diretamente.

Código completo:

```
#include <windows.h>
#include <iostream>
#include <stdio.h>


int main() {

    HMODULE hModule = LoadLibrary(L"user32.dll");
    if (!hModule) {
        std::cerr << "Failed to load user32.dll!" << std::endl;
        return 1;
    }


    typedef int (WINAPI* MsgBoxFunc)(HWND, LPCSTR, LPCSTR, UINT);
    MsgBoxFunc OrdinalBoxA = (MsgBoxFunc)GetProcAddress(hModule,
(LPCSTR)2150);

    if (!OrdinalBoxA) {
        std::cerr << "Failed to locate the function!" << std::endl;
        FreeLibrary(hModule);
        return 1;
    }
```

```
    OrdinalBoxA(NULL, "Hello, World!", "Test MessageBoxA", MB_OK |
MB_ICONINFORMATION);

    FreeLibrary(hModule);
    return 0;
}
```

Como melhoro esse código?

Na perspectiva de Red Team, especialmente quando se considera desenvolvimento de malware e técnicas de evasão, há várias melhorias e técnicas que podem ser incorporadas ao código para aumentar a discrição e dificultar a detecção por soluções de segurança. Aqui estão algumas sugestões para melhorar o código com essas finalidades:

1. Dinamização do Carregamento da DLL

Evitar chamar diretamente funções bem conhecidas como **LoadLibrary** pode ajudar a esquivar-se de ferramentas de monitoramento de comportamento que rastreiam o uso comum de APIs para carregar DLLs. Em vez disso, você pode usar técnicas de execução direta da memória (Reflective DLL Injection) ou carregar a DLL de maneira mais discreta:

- **Carregar a DLL a partir de um local ou recurso não convencional**: Por exemplo, você poderia extrair a DLL de um recurso embutido no executável ou de uma área de dados cifrados.

2. Uso de Syscalls Diretas

Para funções críticas e suscetíveis de serem monitoradas, como a criação de MessageBoxes, considerar a utilização direta de syscalls pode ser uma abordagem mais furtiva. Syscalls não passam diretamente pelas API Wrappers, portanto, são menos propensos a serem capturados por soluções de segurança baseadas em monitoramento de API:

- **Implementar syscalls manualmente**: Isso pode ser feito obtendo o número de syscall correspondente para a função desejada e executando-a diretamente através de assembly in-line no código C++.

3. Obfuscação de Strings e Código

Obfuscar strings (como nomes de DLL e outras constantes de strings) e estruturas de código pode ajudar a evitar detecções baseadas em assinaturas simples:

- **Criptografar strings**: Armazene strings de maneira cifrada e decifre-as em tempo de execução.

- **Técnicas de obfuscação de código**: Alterar a lógica do programa de maneiras que não afetem a funcionalidade, mas modifiquem a aparência do binário.

4. Mudança de Ordinais e Análise Dinâmica

Se possível, não dependa de um ordinal fixo, que pode mudar entre versões ou configurações. Uma análise dinâmica do ambiente pode ajudar a determinar o ordinal correto ou mesmo o nome da função se os ordinais não forem confiáveis:

- **Descobrir ordinais ou nomes em tempo de execução**: Desenvolver um mecanismo que, em tempo de execução, lê a tabela de exportação da DLL e encontra o ordinal correto baseado em heurísticas ou comparações parciais de nomes.

5. Evitando Pontos de API Comuns

Substituir chamadas de API de alto nível por suas contrapartes de mais baixo nível, quando aplicável, ou redirecionar essas chamadas para outras funções menos suspeitas que eventualmente executem a tarefa necessária.

6. Auto-modificação

Considerar técnicas de auto-modificação onde o código altera sua própria execução ou lógica para evitar padrões estáticos que possam ser detectados.

7. Monitoramento e Adaptação ao Ambiente

Detectar a presença de ferramentas de segurança e adaptar o comportamento do código de acordo. Por exemplo, se um sandbox é detectado, o programa pode optar por não executar certas ações ou simular comportamentos benignos.

Essas técnicas requerem um conhecimento profundo dos mecanismos internos do sistema operacional Windows, bem como experiência em programação de baixo nível. Cada uma delas aumenta a complexidade do malware, mas também aumenta suas chances de evadir detecção e realizar suas funções de maneira eficaz e discreta.

Para você praticar em casa:

- Crie um código que chame a API MessageBoxA sem ordinal
- Crie um código que esconda a função MessageBoxA e user32.dll apareçam na tabela IAT

Termos e definições

Termo	Descrição
DLL	Dynamic Link Library (DLL) é um arquivo contendo código e dados que podem ser usados por múltiplos programas simultaneamente.
Linker	Ferramenta que combina vários arquivos de objeto gerados por um compilador em um único executável ou DLL.
Compilador	Programa que traduz código fonte escrito em uma linguagem de programação de alto nível para código de máquina.
MessageBoxA	É uma função da API do Windows que exibe uma caixa de mensagem com texto, botões e ícone personalizáveis, e retorna uma resposta baseada na interação do usuário.
GetProcAddress	É uma função da API do Windows que retorna o endereço de uma função ou variável exportada por uma DLL, identificada pelo seu nome ou ordinal.
Syscall	Um syscall (chamada de sistema) é uma operação fundamental que os programas utilizam para solicitar um serviço do kernel do sistema operacional, como manipulação de arquivos ou comunicação de rede.

Referências Bibliográficas

rioasmara. (2020, November 14). *Hide API Call Strings with Ordinals*. Cyber Security Architect | Red/Blue Teaming | Exploit/Malware Analysis. https://rioasmara.com/2020/11/15/hide-api-call-strings-with-ordinals/#:~:text=The%20ordinal%20represents%20the%20position

GrantMeStrength. (n.d.). *Windows API index - Win32 apps*. Learn.microsoft.com. https://learn.microsoft.com/en-us/windows/win32/apiindex/windows-api-list

NTAPI Undocumented Functions. (n.d.). Undocumented.ntinternals.net. http://undocumented.ntinternals.net/

Yosifovich, P. (2023). Windows Native API Programming. In *leanpub.com*. Leanpub. https://leanpub.com/windowsnativeapiprogramming

Santos, J. A., & Pires, F. (2025). Defense Evasion Techniques: A comprehensive guide to defense evasion tactics for Red Teams and Penetration Testers. In *Amazon*. Packt Publishing. https://www.amazon.com.br/Defense-Evasion-Techniques-comprehensive-Penetration-ebook/dp/B0C5MRV617

Créditos pelo Windows customizado, João Paulo de Andrade